

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN - ĐIỆN TỬ

— * —



BÁO CÁO THỰC TẬP NGOÀI TRƯỜNG
FPGA Implementation of LDPC Decoder
Architecture for Wireless Communication Standards

GVHD : ThS. Nguyễn Trung Hiếu
HỌ VÀ TÊN Vũ Nhật Huy
MSSV : 2311267

Ho Chi Minh City, Ngày 17 tháng 7 năm 2026

Mục lục

1	Giới thiệu	1
1.1	Động lực nghiên cứu	1
1.2	Tình hình nghiên cứu	1
1.3	Cấu trúc báo cáo	2
2	Cơ sở lý thuyết	3
2.1	Tổng quan về Mã Kiểm Tra Chẵn Lẻ Mật Độ Thấp (LDPC)	3
2.2	Thuật Toán Giải Mã Min-Sum	3
2.2.1	Biểu thức rút gọn phục vụ tối ưu hóa phần cứng	5
2.2.2	Ví dụ tổng quát về thuật toán Min-Sum	5
3	Thiết kế và thực hiện phần cứng	7
3.1	Hardware Design	7
4	Thiết kế và thực hiện phần mềm	8
5	Mô phỏng phần cứng	9
6	Kết quả thực hiện	10
7	Kết luận và hướng phát triển	11

Danh sách hình vẽ

Danh sách bảng

1. Giới thiệu

1.1 Động lực nghiên cứu

Mã kiểm tra chẵn lẻ mật độ thấp (Low-Density Parity-Check - LDPC) đóng vai trò quan trọng trong các hệ thống thông tin hiện đại nhờ khả năng sửa lỗi tối ưu. Cụ thể, các nghiên cứu [1] đã chỉ rằng các mã LDPC bất thường nếu được thiết kế tối ưu có thể đạt ngưỡng hiệu suất chỉ cách giới hạn Shannon một khoảng cực kỳ nhỏ là 0.0045 dB ở tỷ lệ lỗi bit $BER = 10^{-6}$. Thậm chí, trong các mô phỏng thực tế với chiều dài khối lên tới 10^7 thì các bộ giải mã này vẫn duy trì khả năng hoạt động cực kỳ ổn định và đạt mức cách giới hạn Shannon chỉ 0.04 dB ở mức $BER = 10^{-6}$. Nhờ đặc tính vượt trội này mà mã LDPC đã trở thành một phần không thể thiếu trong các chuẩn truyền thông không dây băng thông rộng phổ biến như IEEE 802.11n/ad/ay/ax (Wifi), IEEE 802.16e (WiMAX) và mạng di động thế hệ mới ETSI 5G.

Trong thực tế triển khai, các ứng dụng truyền thông hiện đại đòi hỏi tốc độ dữ liệu ngày càng cao và khả năng tương thích linh hoạt với nhiều tiêu chuẩn khác nhau. Tuy nhiên, phần lớn các kiến trúc giải mã song song hiện nay chỉ được thiết kế cố định cho một tiêu chuẩn truyền thông duy nhất hoặc một chiều dài từ mã cụ thể, dẫn đến sự thiếu linh hoạt. Bên cạnh đó, các nghiên cứu trước đây [2], [3], [4] thường lạm dụng tài nguyên bộ nhớ khối (BRAM) để lưu trữ các giá trị nội bộ mà chưa chú trọng đến việc tối ưu hóa cách thức lưu trữ ma trận kiểm tra chẵn lẻ thưa (Sparse Parity-Check Matrix), gây lãng phí tài nguyên phần cứng trên chip. Để giải quyết triệt để, kiến trúc bộ giải mã đề xuất chỉ lưu trữ ma trận kiểm tra chẵn lẻ cơ sở (base parity-check matrix) H_b . Khối dữ liệu này được nén thông qua 3 vector nội bộ: *vals* (chứa các phần tử có giá trị khác -1), *pos* (chứa chỉ số cột tương ứng) và *elementSize* (chứa số lượng phần tử khác -1 trong mỗi hàng). Phương pháp này giúp giảm thiểu đột phá dung lượng BRAM cần thiết.

Việc giải quyết bài toán tối ưu hóa tài nguyên lưu trữ đồng thời tăng cường khả năng cấu hình cho bộ giải mã là vô cùng cấp thiết. Một kiến trúc giải mã QC-LDPC có thể tái cấu hình linh hoạt sẽ giúp giảm thiểu chi phí thiết kế phần cứng và đẩy nhanh tốc độ xử lý. Kết quả của nghiên cứu này có thể ứng dụng trực tiếp vào các hệ thống hạ tầng viễn thông không dây tốc độ cao và các thiết bị lưu trữ thế hệ mới, đáp ứng đồng thời cả hai tiêu chuẩn phổ biến là WiMAX và WiFi trên cùng một nền tảng phần cứng.

1.2 Tình hình nghiên cứu

Trong kỷ nguyên truyền thông tốc độ cao, yêu cầu khắt khe về băng thông đã thúc đẩy vô số công trình nghiên cứu về kiến trúc giải mã LDPC song song được công bố trong thập kỷ qua. Dù các kiến trúc này đã chứng minh được hiệu năng xử lý ở các mức độ nhất định, chúng vẫn tồn tại những nhược điểm về kiến trúc lưu trữ và độ linh hoạt, cụ thể:

- Hạn chế về tính linh hoạt phần cứng. Tiêu biểu như nghiên cứu của C. Kun [2], kiến trúc giải mã được thiết kế cố định cho một khung tiêu chuẩn cụ thể. Điều này đồng nghĩa với việc khi hệ thống cần chuyển đổi từ giao thức WiFi sang WiMAX thì toàn bộ phần cứng phải được thiết kế và tổng hợp lại từ đầu, gây lãng phí thời gian và tốn kém chi phí triển khai.
- Ngoài ra trước đây có khá nhiều thiết kế đã áp dụng phương pháp lưu trữ dữ liệu tính toán nội bộ trực tiếp lên BRAM mà không nén dữ liệu. Nghiên cứu của S.

Mhaske [3] là một ví dụ minh họa rõ nét. Dù kiến trúc giải mã thuật toán này đạt được tốc độ băng thông lên tới $2.48Gb/s$, nhưng lại tiêu tốn một lượng BRAM do không tối ưu hóa việc lưu trữ ma trận thưa. Việc lạm dụng BRAM theo cách truyền thống không chỉ gây lãng phí mà còn làm phức tạp hóa quá trình truy xuất dữ liệu song song, dễ tạo ra các điểm nghẽn băng thông khi mở rộng số lượng lõi giải mã.

- Mặc dù đã có một số ít các công trình nghiên cứu bắt đầu chú ý đến việc tối ưu hóa bộ nhớ, như công trình của S. Nimara [4] đề xuất một kiến trúc giải mã đa từ mã với hiệu suất sử dụng BRAM tốt hơn. Tuy nhiên thì công trình này vẫn thiếu đi khả năng tham số hóa để hỗ trợ đa chuẩn truyền thông băng thông rộng trên cùng một thiết kế nguyên khối. Ngoại trừ số ít nghiên cứu cung cấp được bộ giải mã linh hoạt nhưng phần lớn các kiến trúc hiện hành vẫn chưa giải quyết được bài toán cân bằng giữa tối ưu bộ nhớ và tính linh hoạt đa chuẩn.

Chính từ những nhược điểm này thì việc phát triển một kiến trúc giải mã QC-LDPC kết hợp sử dụng thuật toán Min-Sum và có khả năng cấu hình bằng tham số là vô cùng cấp thiết. Kiến trúc đề xuất trong báo cáo này kỳ vọng giải quyết toàn diện các nhược điểm trên, đạt được băng thông giải mã lên tới $1.2Gbps$ ở tần số hoạt động $240MHz$ mà vẫn giữ mức tiêu thụ tài nguyên phần cứng cực kỳ tối ưu trên FPGA.

1.3 Cấu trúc báo cáo

Tổng quan toàn bộ quá trình thực hiện và triển khai bộ giải mã LDPC của bản báo cáo này được cấu trúc thành 7 chương như sau:

- Chương 1: Giới thiệu.
- Chương 2: Cơ sở lý thuyết.
- Chương 3: Thiết kế và thực hiện phần mềm.
- Chương 4: Thiết kế và thực hiện phần cứng.
- Chương 5: Mô phỏng phần cứng.
- Chương 6: Kết quả thực hiện.
- Chương 7: Kết luận và hướng phát triển.

2. Cơ sở lý thuyết

2.1 Tổng quan về Mã Kiểm Tra Chẵn Lẻ Mật Độ Thấp (LDPC)

LDPC là một lớp mã khối tuyến tính được phát triển ban đầu bởi Gallager vào năm 1963 [5]. Đặc tính tuyến tính này cho phép mã được áp dụng trực tiếp lên các khối dữ liệu nguồn để thực hiện chức năng phát hiện và sửa lỗi trên kênh truyền nhiễu. Dù từng bị lãng quên trong nhiều thập kỷ do giới hạn phần cứng, mã LDPC đã trở lại mạnh mẽ sau khi được MacKay và Neal khám phá lại vào cuối những năm 1990 [6].

Khái niệm mật độ thấp xuất phát từ đặc điểm cấu trúc hình học của ma trận kiểm tra chẵn lẻ H (parity-check matrix) đại diện cho mã. Ma trận H là một ma trận thưa, nghĩa là số lượng phần tử có giá trị bằng '1' chiếm tỷ lệ rất nhỏ so với số lượng phần tử mang giá trị bằng '0' bên trong ma trận.

Một mã khối tuyến tính LDPC đều loại (N, K) được biểu diễn cấu trúc thông qua ma trận kiểm tra H có kích thước $M \times N$, với $M \geq N - K$ hàng và N cột. Khi biểu diễn dưới dạng đồ thị hai phía Tanner, cấu trúc này bao gồm 2 thành phần chính sau:

- Các nút kiểm tra (Check Nodes - CN): Đại diện bởi M hàng của ma trận H , đóng vai trò kiểm tra tính ràng buộc chẵn lẻ của các phương trình toán học.
- Các nút biến (Variable Nodes - VN): Đại diện bởi N cột của ma trận H , tương ứng trực tiếp với các phần tử bit cấu thành nên một từ mã.

Trong các ứng dụng thực tế, các tiêu chuẩn truyền thông không dây băng thông rộng như WiFi và WiMAX không sử dụng mã LDPC mà sử dụng một biến thể gọi là mã LDPC tuần hoàn (QC-LDPC). Đây là nền tảng toán học cốt lõi cho phép triển khai kiến trúc giải mã song song trên phần cứng FPGA. Đặc tính tựa vòng thể hiện ở chỗ ma trận kiểm tra chẵn lẻ H được cấu trúc từ nhiều ma trận con vuông. Các ma trận con này được tạo ra bằng cách dịch vòng một ma trận đơn vị theo các giá trị dịch chuyển xác định.

Nhờ tính chất này, quá trình triển khai phần cứng không cần phải lưu trữ toàn bộ ma trận H khổng lồ. Thay vào đó, bộ giải mã chỉ cần lưu trữ ma trận kiểm tra chẵn lẻ cơ sở H_b (base parity-check matrix). Việc biểu diễn H_b giúp giảm thiểu đáng kể tài nguyên bộ nhớ BRAM cần thiết trên chip. Bên cạnh đó, kích thước của các ma trận con tựa vòng được ký hiệu là hệ số khai triển từ mã z (codeword circulant size), quyết định trực tiếp đến mức độ song song hóa của bộ giải mã. Cụ thể, kiến trúc phần cứng có thể cập nhật đồng thời dữ liệu của z nút biến, cho phép tăng tốc độ thông lượng lên gấp nhiều lần.

2.2 Thuật Toán Giải Mã Min-Sum

Thuật toán giải mã Min-Sum là một phiên bản đơn giản hóa của thuật toán Belief Propagation (BP). Sự đơn giản hóa này được thực hiện bằng cách giảm thiểu tối đa độ phức tạp tính toán tại các khối chức năng chịu trách nhiệm cập nhật nút kiểm tra (Check Node Update - CNU). So với thuật toán BP gốc, việc ứng dụng thuật toán Min-Sum giúp giảm thiểu đáng kể tài nguyên phần cứng tiêu thụ cũng như làm giảm độ phức tạp xử lý toán học của khối CNU [7] mà vẫn duy trì được hiệu suất sửa lỗi ổn định.

Hệ thống giải mã quy ước các đại lượng toán học cơ bản bao gồm:

- c_n : Biểu thị cho bit thứ n của một từ mã phát đi từ nguồn.
- y_n : Giá trị tín hiệu thu được từ kênh truyền ở đầu vào bộ giải mã.

- $r_{mn}[k]$: Thông điệp truyền từ nút kiểm tra m đến nút biến n tại vòng lặp thứ k (thông điệp Check-to-Variable - CTV).
- $q_{mn}[k]$: Thông điệp truyền từ nút biến n đến nút kiểm tra m tại vòng lặp thứ k (thông điệp Variable-to-Check - VTC).
- $N(m)$: Tập hợp các nút biến tham gia vào phương trình kiểm tra của nút kiểm tra m .
- $M(n)$: Tập hợp các nút kiểm tra liên kết trực tiếp với nút biến n .
- $N(m) \setminus n$: Tập hợp các nút biến thuộc $N(m)$ ngoại trừ chính nút biến vị trí n .
- $M(n) \setminus m$: Tập hợp các nút kiểm tra thuộc $M(n)$ ngoại trừ chính nút kiểm tra vị trí m .

Thuật toán giải mã Min-Sum được vận hành tuần tự qua hai giai đoạn chính:

Bước 1: Khởi tạo (Initialization). Tại bước này, giá trị xác suất kênh truyền p_n (thông tin nội tại) của nút biến n được xác định dựa trên tỷ số của xác suất tiên nghiệm:

$$p_n = \log \frac{P(y_n | c_n = 0)}{P(y_n | c_n = 1)} \quad (2.1)$$

Đồng thời, giá trị ban đầu của các thông điệp CTV được thiết lập bằng không: $r_{mn}[0] = 0$.

Bước 2: Giải mã lặp (Iterative Decoding). Tại vòng lặp thứ k , thông điệp VTC là $q_{mn}[k]$ được tổng hợp bằng cách cộng thông tin nội tại của kênh với tổng các thông điệp phản hồi từ các nút kiểm tra liên kết còn lại thông qua công thức:

$$q_{mn}[k] = p_n + \sum_{m' \in \{M(n) \setminus m\}} r_{m'n}[k-1] \quad (2.2)$$

Bộ giải mã tiến hành thực hiện phép quyết định cứng bằng cách tính toán giá trị xác suất hậu nghiệm (A-Posterior Probability - APP) $\Lambda_n[k]$ tại nút biến n hiện tại:

$$\Lambda_n[k] = p_n + \sum_{m' \in M(n)} r_{m'n}[k-1] \quad (2.3)$$

Giá trị của bit kết quả x_n thuộc vectơ từ mã ước lượng sẽ được xác định theo quy tắc:

$$x_n = \begin{cases} 0 & \text{nếu } \Lambda_n[k] > 0 \\ 1 & \text{ngược lại} \end{cases} \quad (2.4)$$

Vòng lặp giải mã sẽ kết thúc hoàn toàn khi vectơ kết quả $x = [x_1, x_2, \dots, x_N]$ thỏa mãn đồng thời tất cả các phương trình kiểm tra chẵn lẻ mang tính tuyến tính của ma trận:

$$H \cdot x = 0 \quad (2.5)$$

Hoặc tiến trình lặp cũng sẽ tự động dừng lại nếu số vòng lặp hiện tại đạt đến giá trị ngưỡng tối đa được thiết lập từ trước trong cấu hình phần cứng hệ thống. Nếu từ mã hợp lệ, chế độ xuất kết quả được kích hoạt để đưa dữ liệu đã sửa lỗi ra ngoài đầu ra.

Nếu phép toán kiểm tra ma trận không thỏa mãn ($H \cdot x \neq 0$), thông điệp kiểm tra phản hồi ngược lại $r_{mn}[k]$ gửi đến nút biến m sẽ được tính toán dựa trên tích các dấu và giá trị cực tiểu của các thông điệp VTC thuộc tập liên kết loại trừ:

$$r_{mn}[k] = \left(\prod_{n' \in \{N(m) \setminus n\}} \text{sign}(q_{mn'}[k]) \right) \times \left(\alpha \times \min_{n' \in \{N(m) \setminus n\}} \{|q_{mn'}[k]|\} \right) \quad (2.6)$$

Trong đó, α là hệ số tỷ lệ điều chỉnh cố định ($\alpha \leq 1$) nhằm bù đắp cho sự suy giảm biên độ do việc đơn giản hóa từ BP sang Min-Sum.

2.2.1 Biểu thức rút gọn phục vụ tối ưu hóa phần cứng

Để tối ưu hóa kiến trúc đọc/ghi song song dữ liệu từ bộ nhớ cổng kép (Dual-port BRAM) trên FPGA, cấu trúc toán học của thuật toán được tinh chỉnh lại. Các phép tính được đưa về quan hệ trực tiếp giữa giá trị APP tổng thể và các thông điệp cập nhật nhằm giảm độ trễ logic.

- Thay vì thực hiện phép cộng tích lũy, VTC được trích xuất trực tiếp bằng cách trừ thông điệp CTV của vòng trước khỏi APP hiện tại:

$$VTC_n = APP_n - CTV_n, \quad n = 1 \dots d_r \quad (2.7)$$

với d_r là trọng số hàng của ma trận kiểm tra.

- Tính toán CTV mới:

$$CTV_{new_n} = \prod_{k=1}^N \text{sign}(VTC_k) \times \text{sign}(VTC_n) \times \min(|VTC|) \quad (2.8)$$

Biểu thức này được hiện thực hóa bằng mạng so sánh logic kết hợp với các cổng logic XOR để xử lý dấu.

- Cập nhật giá trị APP mới cho vòng lặp kế tiếp:

$$APP_{new_n} = VTC_n + CTV_{new_n}, \quad n = 1 \dots d_r \quad (2.9)$$

Kết quả APP mới này sẽ được ghi đè trở lại BRAM, chuẩn bị cho chu kỳ giải mã tiếp theo.

2.2.2 Ví dụ tổng quát về thuật toán Min-Sum

Giả sử ta có từ mã $c = [0 \ 0 \ 1 \ 0 \ 1 \ 1]$, được truyền qua kênh truyền với xác suất lai chéo $p = 0.2$ và giá trị tín hiệu thu được từ kênh truyền $y = [1 \ 0 \ 1 \ 0 \ 1 \ 1]$. [8]

Đầu tiên, ta tiến hành thiết lập ma trận kiểm tra chẵn lẻ H với kích thước 4×6 (4 phương trình kiểm tra và 6 bit từ mã) để tiện quan sát:

$$H = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix} \quad (2.10)$$

Từ 2.1 ta có giá trị xác suất kênh truyền như sau:

$$p_n = \begin{cases} P(y_n|c_n = 0) = \log \frac{p}{1-p}, & \text{nếu } y_i = 1, \\ P(y_n|c_n = 1) = \log \frac{1-p}{p}, & \text{nếu } y_i = 0. \end{cases} \quad (2.11)$$

Như vậy đối với kênh truyền này ta lần lượt có giá trị như sau:

$$P(y_n|c_n = 0) = \log \frac{0.2}{1-0.2} = -1.3863, \quad P(y_n|c_n = 1) = \log \frac{1-0.2}{0.2} = 1.3863 \quad (2.12)$$

Từ đó ta có dãy Log-Likelihood Ratio (LLR) thu được từ kênh truyền như sau:

$$p = [-1.3863, 1.3863, -1.3863, 1.3863, -1.3863, -1.3863] \quad (2.13)$$

Tại vòng lặp 0, các thông điệp VTC chưa qua xử lý, nên sẽ nhận chính giá trị LLR ban đầu này:

$$\begin{aligned} n = 1: & VTC_{1,1} = -1.3863, VTC_{1,3} = -1.3863 \\ n = 2: & VTC_{2,1} = 1.3863, VTC_{2,2} = 1.3863 \\ n = 3: & VTC_{3,2} = -1.3863, VTC_{3,4} = -1.3863 \\ n = 4: & VTC_{4,1} = 1.3863, VTC_{4,4} = 1.3863 \\ n = 5: & VTC_{5,2} = -1.3863, VTC_{5,3} = -1.3863 \\ n = 6: & VTC_{6,3} = -1.3863, VTC_{6,4} = -1.3863 \end{aligned}$$

Tiếp đến, ta cập nhật giá trị CTV thông qua công thức 2.8 ứng với 3 nút kiểm tra:

$$\begin{aligned} m = 1: & CTV_{1,1} = 1.3863, CTV_{1,2} = -1.3863, CTV_{1,4} = -1.3863 \\ m = 2: & CTV_{2,2} = 1.3863, CTV_{2,3} = -1.3863, CTV_{2,5} = -1.3863 \\ m = 3: & CTV_{3,1} = 1.3863, CTV_{3,5} = 1.3863, CTV_{3,6} = 1.3863 \\ m = 4: & CTV_{4,3} = -1.3863, CTV_{4,4} = 1.3863, CTV_{4,6} = -1.3863 \end{aligned}$$

Tiếp theo, ta tiến hành cập nhật giá trị APP mới thông qua 2.9 cho vòng lặp tiếp theo:

$$\begin{aligned} APP_1 &= p_1 + CTV_{1,1} + CTV_{3,1} = -1.3863 + 1.3863 + 1.3863 = 1.3863 \\ APP_2 &= p_2 + CTV_{1,2} + CTV_{2,2} = 1.3863 - 1.3863 + 1.3863 = 1.3863 \\ APP_3 &= p_3 + CTV_{2,3} + CTV_{4,3} = -1.3863 - 1.3863 - 1.3863 = -4.1589 \\ APP_4 &= p_4 + CTV_{1,4} + CTV_{4,4} = 1.3863 - 1.3863 + 1.3863 = 1.3863 \\ APP_5 &= p_5 + CTV_{2,5} + CTV_{3,5} = -1.3863 - 1.3863 + 1.3863 = -1.3863 \\ APP_6 &= p_6 + CTV_{3,6} + CTV_{4,6} = -1.3863 + 1.3863 - 1.3863 = -1.3863 \end{aligned}$$

Căn cứ vào dấu của bộ giá trị APP mới, hệ thống đưa ra phép quyết định cứng dựa vào quy tắc 2.4. Khi ấy ta có từ mã dự đoán:

$$x = [0 \ 0 \ 1 \ 0 \ 1 \ 1] \quad (2.14)$$

Để kiểm tra nếu từ mã x đã đúng hay chưa, ta áp dụng 2.5 và thu được:

$$\mathbf{x}H^T = [0 \ 0 \ 1 \ 0 \ 1 \ 1] \begin{bmatrix} 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} = [0 \ 0 \ 0 \ 0] \quad (2.15)$$

Như vậy x là từ mã hợp lệ.

3. Thiết kế và thực hiện phần cứng

3.1 Hardware Design

In this section, cover the hardware design aspects, including:

- The choice of FPGA platform.
- Design specifications and requirements.
- The hardware architecture and components.
- The design process, including schematic and layout considerations.
- Any simulations or analyses performed to validate the design.

4. Thiết kế và thực hiện phần mềm

Detail the software development aspect of your project. Discuss:

- The programming languages, tools, and frameworks used.
- The design and architecture of your software solution.
- Key algorithms and data structures.
- Any challenges faced and how they were addressed.
- Testing and validation procedures.

5. Mô phỏng phần cứng

This chapter should focus on the simulation aspect of your hardware design. Include:

- The simulation tools and environments used.
- Simulation models and parameters.
- The process of running simulations and analyzing results.
- Any optimizations or modifications made based on simulation outcomes.

6. Kết quả thực hiện

Present the results of your implementation. Discuss:

- The performance metrics and evaluation criteria.
- Results obtained from testing and validation.
- Comparisons with existing solutions or benchmarks.
- Analysis and interpretation of results.
- Any unexpected findings or observations.

7. Kết luận và hướng phát triển

Summarize the key findings and contributions of your thesis. Include:

- A recap of the research problem and objectives.
- Main findings and their implications.
- Limitations of your research.
- Suggestions for future work and potential directions for further research.

Tài liệu tham khảo

- [1] S.-Y. Chung, G. Forney, T. Richardson **and** R. Urbanke, “On the design of low-density parity-check codes within 0.0045 dB of the Shannon limit,” *IEEE Communications Letters*, **jourvol** 5, **number** 2, **pages** 58–60, 2001. DOI: 10.1109/4234.905935
- [2] C. Kun, S. Qi, L. Shengkai **and** P. Chengzhi, “Implementation of encoder and decoder for LDPC codes based on FPGA,” *Journal of Systems Engineering and Electronics*, **jourvol** 30, **number** 4, **pages** 642–650, 2019. DOI: 10.21629/JSEE.2019.04.02
- [3] S. Mhaske, D. Uliana, H. Kee, T. Ly, A. Aziz **and** P. Spasojevic, “A 2.48Gb/s FPGA-based QC-LDPC decoder: An algorithmic compiler implementation,” *in 2015 36th IEEE Sarnoff Symposium* 2015, **pages** 88–93. DOI: 10.1109/SARNOF.2015.7324649
- [4] S. Nimara, O. Boncalo, A. Amaricai **and** M. Popa, “FPGA architecture of multi-codeword LDPC decoder with efficient BRAM utilization,” *in 2016 IEEE 19th International Symposium on Design and Diagnostics of Electronic Circuits Systems (DDECS)* 2016, **pages** 1–4. DOI: 10.1109/DDECS.2016.7482452
- [5] R. G. Gallager, “Low-Density Parity-Check Codes,” Cambridge, MA: M.I.T. Press, 1963.
- [6] D. MacKay **and** R. Neal, “Near Shannon limit performance of low density parity check codes,” *Electronics Letters*, **jourvol** 32, **pages** 1645–1646, 18 1996. DOI: 10.1049/el:19961141 eprint: <https://digital-library.theiet.org/doi/pdf/10.1049/el%3A19961141>. url: <https://digital-library.theiet.org/doi/abs/10.1049/el%3A19961141>
- [7] K. Zhang, X. Huang **and** Z. Wang, “High-throughput layered decoder implementation for quasi-cyclic LDPC codes,” *IEEE J.Sel. A. Commun.*, **jourvol** 27, **number** 6, **pages** 98–994, **august** 2009, ISSN: 0733-8716. DOI: 10.1109/JSAC.2009.090816 url: <https://doi.org/10.1109/JSAC.2009.090816>
- [8] S. J. Johnson, “Introducing Low-Density Parity-Check Codes,” 2008, **page** 49. url: <https://api.semanticscholar.org/CorpusID:17943867>